DISCRETE MATHEMATICS

1. This question carries 30% of the marks.

   a) For each of the following statements, state whether it is true or false and provide a supporting proof.

      i) $n^2 + n = \omega(n^2)$. [3]

      Answer:
      False. LHS/RHS gives $(n^2+n)/n^2 \to 1$ as $n \to \infty$ (it should tend to infinity for the statement to be true).

      ii) $(n^2+1)/(n^2-1) = O(n)$. [3]

      Answer:
      True. The LHS tends to 1 for large n, so it's less than n.

      iii) If $f(n) = \omega(n)$ then $1/f(n) = o(1/n)$. [3]

      Answer:
      True. If $f(n) = \omega(n)$ then $f(n)/n \to \infty$ which means $(1/f(n))/(1/n) = n/f(n) \to 0$.

   b) Give a tight bound for each of the following recurrence relations, or explain why it is not possible to do so.

      i) $T(n) = 27\,T(n/3) + n^3$. [2]

      Answer:
      $a = 27, b = 3, d = 3$ so $\log_b a = \log_3 27 = 3 = d$ so $T(n) = O(n^d \log n) = O(n^3 \log n)$.

      ii) $T(n) = 26\,T(n/3) + n^3$. [2]

      Answer:
      $a = 26, b = 3, d = 3$ so $\log_b a = \log_3 26 < \log_3 27 = 3 = d$ so $T(n) = O(n^d) = O(n^3)$.

      iii) $T(n) = 2\,T(n/16) + 4$. [2]

      Answer:
      $a = 2, b = 16, d = 0$ so $\log_b a = \log_{16} 2 = 1/4 > d$ so $T(n) = O(n^{1/4})$.

Master Theorem. If $T(n)$ satisfies

$$T(n) = a\,T(\,n/b\,) \;+\; O(n^d)$$

for some $a > 0$, $b > 1$ and $d \geq 0$, then

$$T(n) = \begin{cases} O(n^d) & \text{if } d > \log_b a \\ O(n^d \log n) & \text{if } d = \log_b a \\ O(n^{\log_b a}) & \text{if } d < \log_b a \end{cases}$$

Answers

2. This question carries 20% of the marks.

The following code is an implementation of a queue data structure. A new item is added to the queue by calling `enqueue(item)`. Calling `dequeue()` removes and returns the first remaining item that was added to the queue. The queue is implemented with list data structures that have an `append(item)` method that adds an item to the end of the list, and a `pop()` method that removes and returns the last item of the list. You can assume both operations are $O(1)$.

```
1.  in = empty list
2.  out = empty list
3.  method enqueue(item)
4.      in.append(item)
5.  method dequeue()
6.      if length(out)==0
7.          while length(in)>0
8.              out.append(in.pop())
9.      return out.pop()
```

a) Explain briefly in words how the algorithm ensures that the `dequeue()` method returns items in the order they were added by `enqueue(item)`. [2]

Answer:

The `in` list stores items in the order they were added (L4), but you can only get items from the end, which would be reverse order. So we have a second list `out` that will store items in reverse order. We can then pop items from this list and it will be the right order (L9). When `out` has been emptied (L6), we copy the whole of `in` to `out` reversing the order (L7-8), because the last item of `in` becomes the first of `out` and so on.

b) Calculate the worst case complexity of the `enqueue(item)` method. Assume that `in` has n items. [1]

Answer:

L4 is just an append which is stated in the question to be $O(1)$.

c) Calculate the worst case complexity of the `dequeue()` method. Assume that `in` has n items. [2]

Answer:

In the worst case, `length(out)==0` and L7-8 have to run. Since `in` has n items the loop is executed n times and each iteration is $O(1)$ so the total cost is $O(n)$. The final operation on L8 is $O(1)$ and so the worst case for `dequeue()` is $O(1)+O(n)=O(n)$.

d) Assume that `out` is empty, `in` has n items and `dequeue()` has just been called. Compute the complexity of the worst case total amount of time spent in the `dequeue` method from now until the next time the `out` list is empty. Carefully justify your answer. [4]

Answer:

The first call that is made is to `dequeue()` and the `out` array is empty, so the n items from `in` are copied, at a cost of $O(n)$. From now until the `out` array is empty, every call to `dequeue()` will only be $O(1)$ because the condition `length(out)==0` will not be true. There may be many calls to `enqueue(item)` in this time, but since we are not counting them, and the subsequent calls to `dequeue()` don't depend on the number of items in `in`, we can ignore this. Since the length of `out` decreases by 1 each time, we have that after the first call to `dequeue()`, n items are copied into `out` but one is immediately popped, leaving $n-1$ items in `out`. This decreases by 1 each call until we get to 0, so

Answers

the total cost is 1 call that costs $O(n)$ and $n-1$ calls that cost $O(1)$ each, at a total cost of $O(n)+(n-1)O(1)=O(n)$.

e) Calculate the amortised cost of the `dequeue()` method. [1]

Answer:

The amortised cost is the total cost divided by the number of calls in the previous part, so $O(n)/n = O(1)$.

Answers

3. This question carries 50% of the marks.

The museum thief has done her research, and found that at the museum there are N items she would like to steal. Item $i = 1, \ldots, N$ could be sold for a price p_i, has weight w_i and would take her a total time t_i to wrap up safely so that it doesn't get damaged. She estimates she can carry a total weight W and that the alarms will go off after time T. She sets out to design an algorithm to maximise the amount of money she can make.

a) Write $P(n,w,t)$ to be the maximum amount of money she can make if she only chose from items $i = 1, \ldots, n \leq N$ with a maximum total weight of $w \leq W$ and time $t \leq T$. Assume that all weights and times are integer valued.

i) By considering the choice as to whether or not to take item n, compute a recurrence relation for $P(n,w,t)$ in terms of the values of $P(n',w',t')$ for $n' \leq n$, $w' \leq w$ and $t' \leq t$. Remember to consider the base cases. [6]

Answer:
If any of n, w or t is 0, then no items can be taken so $P(n,w,t) = 0$. This is the base case. If $w_n > w$ or $t_n > t$ then it would be too heavy or take too long to pack, so this item cannot be taken and therefore the best price is the same as the best price for the same weights and times but with the first $n-1$ items. So $P(n,w,t) = P(n-1,w,t)$ in this case. Otherwise, we consider the two options. Option 1 is not to take it, in which case $P(n,w,t) = P(n-1,w,t)$. Option 2 is to take it, but this uses up weight w_n leaving a remaining capacity of $w - w_n$, and time t_n leaving a remaining time of $t - t_n$. So in this case $P(n,w,t) = p_n + P(n-1, w-w_n, t-t_n)$. We take the maximum of these two. In summary:

$$P(n,w,t) = \begin{cases} 0 & \text{if } n = 0,\ w = 0 \text{ or } t = 0 \\ P(n-1,w,t) & \text{if } w_n > w \text{ or } t_n > t \\ \max\{P(n-1,w,t),\ p_n + P(n-1, w-w_n, t-t_n)\} & \text{otherwise} \end{cases}$$

Answers

ii) Write pseudocode for an algorithm based on this recurrence relation that returns the optimal set of items to take. [6]

Answer:
The following Python pseudocode works:

```
def optimal_dp(pi, wi, ti, W, T):
    memo = {}
    def P(n, w, t):
        if n==0 or w==0 or t==0:
            return 0, [] # no items
        if (n, w, t) not in memo:
            if wi[n-1]>w or ti[n-1]>t:
                memo[n, w, t] = P(n-1, w, t)
            else:
                # leave it
                P0, S0 = P(n-1, w, t)
                # take it
                P1, S1 = P(n-1, w-wi[n-1], t-ti[n-1])
                P1 = P1+pi[n-1]
                S1 = S1+[n-1]
                # take the best
```

```
                    if P0>P1:
                        memo[n, w, t] = (P0, S0)
                    else:
                        memo[n, w, t] = (P1, S1)
                return memo[n, w, t]
            Po, So = P(len(pi), W, T)
            return So
```

iii) Compute the complexity of this algorithm in terms of N, W and T. [3]

Answer:
There are N values n can take, W values w can take and T values t can take, and in the worse case we might need to compute $P(n,w,t)$ for all of these. Each function call is $O(1)$ ignoring the recursive calls and values are not recomputed because of the memoisation, so the total time is $O(NWT)$.

b) The museum thief is – perhaps unsurprisingly – greedy, and so she wants an algorithm that is less work. She proposes to just start with the most expensive item and keep going down the list in order of decreasing price, discarding items if she can't carry that much weight or doesn't have enough time remaining.

i) Write down an inequality for the price p_n in terms of the prices p_i for $i < n$ that guarantees that this algorithm gives the optimal choice for whether or not to take item n, whatever the values of w_i and t_i. Prove that your inequality is correct. [5]

Answer:
In order to guarantee that we can take item n without thinking about items 1 to $n-1$, we have to ensure that the price for item n is higher than the sum of the prices of items 1 to $n-1$, otherwise taking item n might stop us from taking a more advantageous sum of items from 1 to $n-1$. In other words, $p_n \geq p_1 + \cdots + p_{n-1}$. We can prove this guarantees the correctness of the algorithm as follows. Suppose $I \subseteq \{1,\ldots,n-1\}$ is the optimal choice not taking item n, and $J \subseteq \{1,\ldots,n\}$ (with $n \in J$) is the optimal choice where we do take item n. We would have to have that $\sum_{i\in I} p_i > \sum_{i\in J} p_i = p_n + \sum_{i\in I-\{n\}} p_i \geq p_n$, but this gives us the inequality $\sum_{i=1}^{n-1} p_i \geq \sum_{i\in I} p_i > p_n$ contradicting our assumption.

ii) Show that if the inequality is false for some n, there is a set of weights w_i and times t_i that make her algorithm give a choice that is not optimal. [3]

Answer:
Take $w_1 = \cdots = w_{n-1} = t_1 = \cdots = t_{n-1} = 1$, $w_n = t_n = W = T = n$. In this case, the algorithm will choose to take item n, but taking items 1 to $n-1$ is also feasible and has a higher sum of prices (if the inequality is false).

iii) Assuming $p_1 = 1$ what is the smallest set of prices for which the greedy algorithm is guaranteed to be optimal? [2]

Answer:
The smallest price for p_2 satisfying the inequality must have $p_2 \geq p_1$ so we can set $p_2 = p_1 = 1$. The smallest price for $p_3 = p_1 + p_2 = 2$, for $p_4 = p_3 + p_2 + p_1 = 4$ and so on. So $p_n = 2^{n-2}$ for $n \geq 2$.

Answers